

SPEC TÉCNICA v2

EffectusHub - Integração Meta Ads (Facebook e Instagram)

Documento de implementação para desenvolvimento | Node.js | Sandbox → Produção

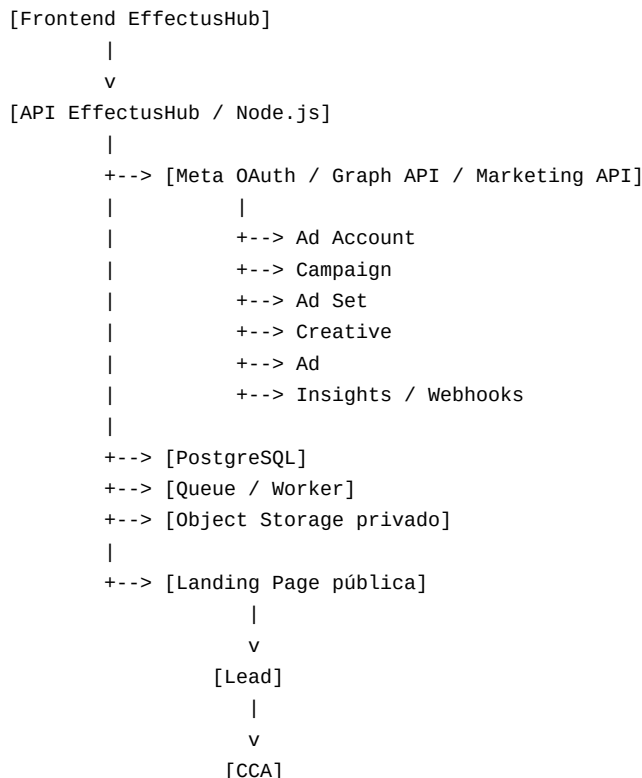
Objetivo: permitir que corretores/empresas criem e acompanhem campanhas de Facebook e Instagram diretamente no EffectusHub, vinculadas a imóvel, corretor e CCA, com captura de leads e encaminhamento automático para análise de crédito.

Segurança: nenhum Access Token real, App Secret ou credencial fornecida durante os testes foi incluído neste documento. O token exibido durante o processo de configuração deve ser revogado/substituído antes de uso real.

1. Escopo funcional

- Conectar uma conta Meta ao EffectusHub via OAuth.
- Listar e selecionar contas de anúncios às quais o usuário autorizado possui acesso.
- Criar campanhas para Facebook/Instagram a partir de um imóvel cadastrado no CRM.
- Configurar objetivo, orçamento, período, público, posicionamentos, criativo e URL de destino.
- Criar a estrutura Meta: Campaign → Ad Set → Ad Creative → Ad.
- Publicar inicialmente em PAUSED e ativar somente após validações internas.
- Registrar IDs externos e sincronizar status/métricas.
- Capturar lead em landing page do EffectusHub, incluindo documentos em armazenamento privado.
- Associar lead automaticamente a campanha, imóvel, corretor e CCA selecionada.
- Encaminhar o lead para a CCA por fila assíncrona e acompanhar análise de crédito.
- Manter rastreabilidade financeira e de auditoria sem intermediar o pagamento da mídia no MVP.

2. Arquitetura proposta



Regra de arquitetura: o frontend nunca deve falar diretamente com a Meta usando App Secret ou token persistente. O fluxo deve ser Frontend → Backend EffectusHub → Meta.

3. SDK e dependências Node.js

O sample gerado pela própria Meta usa o pacote `facebook-nodejs-business-sdk` e instancia `AdAccount` para criar Campaign. Essa será a referência inicial para o provider Meta do EffectusHub.

```
npm install facebook-nodejs-business-sdk
const bizSdk = require('facebook-nodejs-business-sdk');
const { FacebookAdsApi, AdAccount } = bizSdk;
```

Observação do sample: o código original fornecido declara `ad\_account\_id`, mas chama `new AdAccount(account\_id)`. Na implementação do EffectusHub, padronizar para `adAccountId` e nunca depender de variável global ambígua.

4. Configuração e variáveis de ambiente

Variável	Uso
META_APP_ID	ID do aplicativo Meta.
META_APP_SECRET	Segredo do aplicativo. Somente backend/secret manager.
META_REDIRECT_URI	Callback OAuth do EffectusHub.
META_GRAPH_API_VERSION	Versão da Graph API configurada/validada pelo time.
META_WEBHOOK_VERIFY_TOKEN	Token interno para verificação do webhook.
META_WEBHOOK_SECRET	Segredo interno para validações adicionais, se aplicável.
META_TOKEN_ENCRYPTION_KEY	Chave usada para criptografar tokens persistidos.

```
META_APP_ID=...
META_APP_SECRET=...
META_REDIRECT_URI=https://app.effectushub.com.br/api/integrations/meta/callback
META_GRAPH_API_VERSION=vXX.X
META_WEBHOOK_VERIFY_TOKEN=...
META_TOKEN_ENCRYPTION_KEY=...
```

Não fixar a versão da Graph API no código-fonte. A versão deve ser configurável e atualizada após validação de compatibilidade.

5. Permissões Meta

No fluxo de teste do app, as permissões observadas como disponíveis incluem `ads\_read`, `ads\_management`, `business\_management`, `pages\_read\_engagement` e `pages\_show\_list`. Solicitar em produção somente o conjunto mínimo necessário ao fluxo efetivamente implementado.

Permissão	Finalidade no EffectusHub
ads_read	Consultar contas, campanhas, anúncios, status e métricas permitidas.
ads_management	Criar/alterar campanhas, conjuntos, criativos e anúncios.
business_management	Acessar ativos comerciais quando necessário ao fluxo.
pages_show_list	Listar Páginas acessíveis quando necessário à seleção do ativo.
pages_read_engagement	Leitura de dados de Página necessários ao fluxo, quando aplicável.

Antes da publicação do app, validar no painel da Meta quais permissões exigem análise, acesso avançado e demonstração de uso. Não assumir que o estado de Sandbox equivale a autorização de produção.

6. OAuth e conexão da conta Meta

6.1 Endpoints internos

Método	Endpoint	Descrição
GET	/api/integrations/meta/connect	Gera URL OAuth com state assinado e redireciona o usuário.
GET	/api/integrations/meta/callback	Valida state, recebe code e troca por token

		no backend.
GET	/api/integrations/meta/ad-accounts	Lista contas de anúncios disponíveis.
PUT	/api/integrations/meta/ad-account	Define a conta escolhida para o tenant/usuário.
DELETE	/api/integrations/meta	Desconecta a integração e invalida o vínculo local.

6.2 Regras

- Usar `state` criptograficamente aleatório/assinado para proteção contra CSRF.
- Troca de `code` por token deve ocorrer exclusivamente no backend.
- Criptografar token em repouso; nunca registrar token completo em logs.
- Validar que a conta de anúncios escolhida pertence ao conjunto retornado pela Meta para o usuário.
- Modelar conexão por tenant/empresa; não compartilhar credenciais entre clientes.

7. Modelo de dados

7.1 meta_connections

Campo	Tipo/observação
id	UUID PK
tenant_id	UUID, indexado
user_id	UUID que realizou a conexão
meta_user_id	string
access_token_encrypted	text cifrado
token_expires_at	timestamp nullable
ad_account_id	string, ex.: act_xxx
page_id	string nullable
instagram_actor_id	string nullable
scopes	jsonb
status	CONNECTED EXPIRED REVOKED ERROR
created_at / updated_at	timestamps

7.2 campaigns

Campo	Tipo/observação
id	UUID PK
tenant_id	UUID
property_id	UUID obrigatório
broker_id	UUID obrigatório
cca_id	UUID obrigatório para roteamento
provider	META
name	string
objective	string
daily_budget / lifetime_budget	numeric nullable
start_at / end_at	timestamp nullable
status	DRAFT READY PUBLISHING ACTIVE PAUSED FINISHED REJECTED ERROR ARCHIVED
meta_campaign_id	string nullable
meta_adset_id	string nullable
meta_creative_id	string nullable
meta_ad_id	string nullable
public_token	token não adivinhável para landing page
last_sync_at	timestamp nullable

7.3 campaign_publish_attempts

Campo	Descrição
id	UUID PK
campaign_id	FK
idempotency_key	chave única por tentativa lógica
step	CAMPAIGN ADSET CREATIVE AD
request_summary	json sem segredos
provider_object_id	ID criado na Meta, se houver

status	STARTED SUCCESS FAILED
error_code / error_message	erro sanitizado
created_at	timestamp

7.4 leads e documentos

Tabela	Campos principais
leads	id, tenant_id, campaign_id, property_id, broker_id, cca_id, nome, email, telefone, cpf_hash/opcional, source, utm_*, status, created_at
lead_documents	id, lead_id, storage_key, original_name, mime_type, size, malware_scan_status, created_at
lead_consent	id, lead_id, consent_type, text_version, accepted_at, ip_hash, user_agent

8. API interna de campanhas

Método	Endpoint	Função
POST	/api/campaigns	Cria rascunho ligado a imóvel/corretor/CCA.
PUT	/api/campaigns/:id	Atualiza configuração enquanto permitido.
PUT	/api/campaigns/:id/creative	Atualiza texto, mídia, CTA e URL.
POST	/api/campaigns/:id/publish	Dispara pipeline assíncrono de publicação.
POST	/api/campaigns/:id/pause	Pausa campanha/anúncios na Meta.
POST	/api/campaigns/:id/resume	Retoma conforme regras da Meta.
GET	/api/campaigns/:id	Detalhes e IDs externos.
GET	/api/campaigns/:id/metrics	Métricas sincronizadas.
GET	/api/campaigns	Lista por tenant com filtros.

9. Provider abstraction

```

class AdvertisingProvider {
  async listAdAccounts(connection) {}
  async createCampaign(input) {}
  async createAdSet(input) {}
  async createCreative(input) {}
  async createAd(input) {}
  async pauseCampaign(externalId) {}
  async resumeCampaign(externalId) {}
  async getInsights(input) {}
}

class MetaAdsProvider extends AdvertisingProvider {
  // implementação usando facebook-nodejs-business-sdk
}

```

Esse contrato evita acoplar o domínio do EffectusHub à Meta e permite adicionar Google Ads/TikTok Ads posteriormente.

10. Criação de Campaign - código base adaptado

O sample oficial fornecido cria uma campanha com objetivo `OUTCOME\_TRAFFIC`, status inicial `PAUSED` e `special\_ad\_categories: []`. No EffectusHub, os valores devem vir de configuração validada e não de constantes espalhadas pelo código.

```

const bizSdk = require('facebook-nodejs-business-sdk');
const { FacebookAdsApi, AdAccount } = bizSdk;

async function createMetaCampaign({
  accessToken,
  adAccountId,
  campaignName,

```

```

    objective = 'OUTCOME_TRAFFIC',
    specialAdCategories = [],
  }) {
    if (!accessToken || !adAccountId || !campaignName) {
      throw new Error('Missing required Meta campaign parameters');
    }

    const api = FacebookAdsApi.init(accessToken);

    // DEBUG da SDK deve permanecer desligado em produção para evitar
    // exposição accidental de payloads/credenciais em logs.
    api.setDebug(false);

    const account = new AdAccount(adAccountId);

    const campaign = await account.createCampaign([], {
      name: campaignName,
      objective,
      status: 'PAUSED',
      special_ad_categories: specialAdCategories,
    });

    return {
      campaignId: campaign.id,
    };
  }

  module.exports = { createMetaCampaign };

```

10.1 Regras adicionais

- Nunca salvar Access Token no código, `.env` versionado ou banco sem criptografia.
- Nunca logar headers Authorization, token ou App Secret.
- Criar em `PAUSED` no MVP; ativação deve ser uma ação explícita e auditável.
- Persistir `meta\_campaign\_id` imediatamente após sucesso para permitir recuperação parcial.
- Antes de usar `special\_ad\_categories: []`, o backend deve determinar se a campanha exige categoria especial segundo as regras vigentes da Meta. Não assumir lista vazia para todo anúncio imobiliário.
- Objetivo e campos aceitos devem ser validados contra a versão vigente da Marketing API antes da publicação.

11. Pipeline de publicação

1. Validar tenant, usuário, imóvel, CCA e integração Meta.
2. Validar conta de anúncios e ativos necessários (Página/Instagram quando aplicável).
3. Validar criativo, URL pública, orçamento, datas, público e categoria especial.
4. Marcar campanha interna como PUBLISHING e gerar `idempotency\_key`.
5. Criar Campaign em PAUSED; persistir `meta\_campaign\_id`.
6. Criar Ad Set; persistir `meta\_adset\_id`.
7. Criar/associar mídia e Ad Creative; persistir `meta\_creative\_id`.
8. Criar Ad; persistir `meta\_ad\_id`.
9. Consultar objetos criados e confirmar consistência.
10. Marcar como PAUSED/READY no CRM; ativar apenas após comando explícito ou regra aprovada.

Falha intermediária não deve apagar IDs já criados. O worker deve ser reentrante: se Campaign já existe, retomar do próximo passo em vez de duplicá-la.

12. Ad Set, público e posicionamentos

O Ad Set concentra orçamento quando aplicável, período, otimização, público e posicionamentos. O frontend deve trabalhar com opções de negócio simples e o backend convertê-las para parâmetros Meta.

- Localização: cidade/região/raio conforme opções suportadas.
- Faixa etária e demais segmentações somente quando permitidas para a categoria do anúncio.
- Posicionamentos: automático como padrão inicial; permitir customização somente após validação.
- Facebook/Instagram: selecionar ativos vinculados e elegíveis.
- Orçamento: armazenar internamente em unidade monetária e converter para a unidade exigida pela API.

Imóveis podem estar sujeitos às regras de categorias especiais da Meta. O time deve validar as regras vigentes antes de permitir segmentações que possam ser restringidas.

13. Criativo e anúncio

13.1 Dados internos

- headline, primary_text, description, call_to_action
- media_asset_id / image_hash / video_id conforme estratégia
- destination_url
- page_id
- instagram_actor_id, quando aplicável

13.2 URL e tracking

```
https://app.effectushub.com.br/i/{publicToken}
?utm_source=meta
&utm_medium=paid_social
&utm_campaign={internalCampaignId}
&utm_content={creativeId}
```

Além de UTMs, a landing page deve resolver o `publicToken` no backend e recuperar de forma segura campanha, imóvel, corretor e CCA. Nunca confiar apenas nos parâmetros UTM para vínculo de negócio.

14. Landing page, lead e documentos

14.1 Endpoint público

```
POST /public/campaigns/{publicToken}/leads
Content-Type: multipart/form-data
```

14.2 Fluxo

11. Validar `publicToken`, campanha e disponibilidade do imóvel.
12. Aplicar rate limit, anti-bot e validações de entrada.
13. Criar lead com origem META e IDs internos.
14. Detectar duplicidade por regras de telefone/email/documento.
15. Fazer upload dos documentos diretamente para storage privado via URL assinada ou backend controlado.
16. Executar validação de tipo/tamanho e malware scan.
17. Registrar consentimento e versão do texto LGPD.
18. Enfileirar roteamento para a CCA selecionada.
19. Retornar confirmação sem expor dados internos da CCA.

15. Roteamento automático para CCA

```
LeadCreated
|
```

```

v
Queue: lead.route-to-cca
|
+--> valida lead/documentos
+--> resolve cca_id da campanha
+--> cria solicitação de análise
+--> envia dados permitidos
+--> registra protocolo/status
+--> atualiza CRM

```

O roteamento deve ser assíncrono, idempotente e auditável. Falha na CCA não deve perder o lead; deve gerar retry com backoff e, após limite, encaminhar para fila de erro/atendimento manual.

16. Webhooks Meta

Método	Endpoint	Uso
GET	/webhooks/meta	Handshake/verificação exigida pela Meta.
POST	/webhooks/meta	Recebe eventos configurados e os envia para fila.

- Validar autenticidade conforme mecanismo vigente da Meta.
- Responder rapidamente ao webhook; processamento pesado deve ocorrer em worker.
- Deduplicar eventos por identificador/hash.
- Nunca confiar no webhook como única fonte de verdade; reconciliar periodicamente com API.
- Guardar payload sanitizado pelo período necessário à auditoria.

17. Insights e dashboard

Sincronizar métricas periodicamente para reduzir chamadas em tempo real e permitir histórico no CRM.

Métrica	Exibição sugerida
spend	Valor gasto
impressions	Impressões
reach	Alcance
clicks / link clicks	Cliques
ctr	CTR
cpc	CPC
leads internos	Leads capturados no EffectusHub
cost_per_internal_lead	spend / leads internos, calculado no CRM

As métricas disponíveis e seus nomes podem variar por versão/objetivo. O adapter deve mapear a resposta da Meta para métricas internas estáveis.

18. Pagamento e orçamento

MVP recomendado: o pagamento da mídia permanece na conta de anúncios Meta conectada pelo cliente. O EffectusHub configura orçamento e acompanha gasto, mas não recebe nem repassa o valor da mídia.

- Exibir claramente que orçamento da campanha não é cobrança do EffectusHub.
- Não armazenar cartão ou dados de pagamento da Meta.
- Bloquear publicação se a conta não estiver apta segundo a resposta da Meta.
- Registrar orçamento solicitado, moeda e gasto sincronizado para auditoria.

19. Segurança e LGPD

- HTTPS obrigatório.
- Tokens e segredos em secret manager/criptografia forte; rotação suportada.
- RBAC por tenant e princípio de menor privilégio.
- Isolamento de tenant em todas as queries e jobs.

- CSRF no OAuth; state com expiração e uso único.
- Rate limiting em endpoints públicos e sensíveis.
- Uploads privados; URLs assinadas com curta expiração.
- Validação MIME por conteúdo, tamanho máximo e malware scan.
- Logs sem PII desnecessária e sem credenciais.
- Consentimento explícito e versionado para tratamento/compartilhamento de dados conforme finalidade.
- Política de retenção e exclusão para leads/documentos.
- Auditoria de conexão, publicação, pausa, alteração de orçamento e acesso a documentos.

20. Erros e observabilidade

Categoria	Tratamento
AUTH_EXPIRED	Marcar conexão como EXPIRED e solicitar reconexão.
PERMISSION_DENIED	Registrar escopo/ação; não repetir indefinidamente.
RATE_LIMIT	Retry com backoff/jitter e limites.
VALIDATION_ERROR	Retornar erro de negócio ao frontend com campo acionável.
META_REJECTED	Persistir motivo/status; exibir sem alterar a mensagem original indevidamente.
NETWORK/TIMEOUT	Retry idempotente.
PARTIAL_PUBLISH	Retomar do último objeto externo persistido.

- Correlation ID por requisição/job.
- Métricas: taxa de publicação, latência, falhas por etapa, retries, webhooks processados.
- Alertas para aumento de falhas de autenticação, rate limit ou publicação.

21. Sandbox e estratégia de testes

A conta de anúncios Sandbox permite testar criação e relatórios sem cobrança e sem veiculação real. Usá-la como primeira etapa obrigatória.

21.1 Testes mínimos

- OAuth/conexão e expiração/revogação.
- Listagem e seleção de ad account.
- Criação de Campaign em PAUSED.
- Idempotência: repetir publish não duplica Campaign.
- Falha após Campaign e retomada em Ad Set.
- Validação de orçamento, datas e campos obrigatórios.
- Criação de landing page e lead.
- Upload inválido/malicioso bloqueado.
- Deduplicação de lead.
- Roteamento CCA com sucesso, retry e dead-letter.
- Sincronização de status/métricas.
- Tenant A não acessa dados/credenciais do Tenant B.

22. Passagem para produção

20. Concluir implementação e testes na Sandbox.
21. Configurar domínio, URLs de callback, política de privacidade e exclusão de dados.
22. Revisar permissões mínimas realmente utilizadas.
23. Preparar evidências/demonstração para análise da Meta quando exigido.
24. Solicitar os níveis de acesso necessários no painel do app.
25. Testar com conta real controlada e campanha em PAUSED.
26. Validar Página e Instagram vinculados.
27. Ativar monitoramento, limites e alertas.
28. Executar rollout gradual por tenant.

Não publicar o app ou campanhas reais usando o token exposto durante a configuração inicial. Gerar/revogar credenciais conforme o fluxo oficial antes do go-live.

23. Critérios de aceite técnicos

- Usuário autorizado consegue conectar sua conta Meta sem informar senha ao EffectusHub.
- Backend lista somente contas de anúncios às quais a conexão possui acesso.
- Campaign é criada via API e o `meta\_campaign\_id` é persistido.
- Pipeline cria Ad Set, Creative e Ad sem duplicação em retries.
- Campanha nasce PAUSED no MVP.
- Facebook/Instagram são selecionados apenas quando os ativos necessários estão válidos.
- Lead originado da landing page é ligado inequivocamente à campanha, imóvel, corretor e CCA.
- Documentos não ficam públicos e possuem controles de acesso.
- CCA recebe o lead de forma assíncrona e auditável.
- Dashboard apresenta status e métricas sincronizadas.
- Tokens/segedos não aparecem em frontend, logs ou código-fonte.
- Falhas da Meta são tratadas sem perda de estado.
- Existe trilha de auditoria das ações críticas.
- Fluxo Sandbox é aprovado antes de qualquer ativação real.

24. Ordem recomendada de implementação

Sprint/Etapa	Entrega
1	MetaAdsProvider + configuração segura + OAuth + meta_connections.
2	Listagem/seleção de ad account + Campaign em Sandbox.
3	Ad Set + Creative + Ad + persistência de IDs + idempotência.
4	Landing page + tracking + lead + documentos.
5	Fila/integração CCA + status de análise.
6	Webhooks + Insights + dashboard.
7	Hardening de segurança/LGPD + observabilidade + testes E2E.
8	App Review/acesso de produção + piloto controlado.

25. Exemplo de serviço consolidado

```
class MetaAdsService {
  constructor({ connectionRepository, campaignRepository, cryptoService }) {
    this.connectionRepository = connectionRepository;
    this.campaignRepository = campaignRepository;
    this.cryptoService = cryptoService;
  }

  async publishCampaign({ tenantId, campaignId }) {
    const campaign = await this.campaignRepository.getForTenant(
      tenantId,
      campaignId
    );

    const connection = await this.connectionRepository.getActiveForTenant(
      tenantId
    );

    const accessToken = await this.cryptoService.decrypt(
      connection.access_token_encrypted
    );
  }
}
```

```
// 1. Validar configuração e ativos.  
// 2. Criar/recuperar Campaign.  
// 3. Criar/recuperar Ad Set.  
// 4. Criar/recuperar Creative.  
// 5. Criar/recuperar Ad.  
// 6. Persistir IDs após cada etapa.  
// 7. Manter PAUSED até ativação explícita.  
}  
}
```

26. Definition of Done

- Código revisado e sem credenciais hardcoded.
- Testes unitários, integração e E2E críticos aprovados.
- Sandbox cria a estrutura completa sem duplicações.
- Documentação de configuração e rotação de credenciais pronta.
- Runbook de falhas Meta/CCA disponível.
- LGPD e controles de acesso revisados.
- Observabilidade e alertas ativos.
- Critérios de aceite funcionais e técnicos aprovados.
- Produção somente após permissões/acessos Meta necessários estarem aprovados.

27. Referência do sample fornecido

O arquivo de exemplo fornecido pelo usuário foi gerado para Node.js com `facebook-nodejs-business-sdk`. Ele inicializa `FacebookAdsApi`, cria uma Campaign por `AdAccount.createCampaign()`, usa objetivo `OUTCOME\_TRAFFIC`, status `PAUSED` e imprime o ID retornado. A SPEC adapta esse padrão para arquitetura multi-tenant, segurança, idempotência e persistência do EffectusHub.

Importante: o Access Token existente no sample original não foi reproduzido neste documento e deve ser revogado/substituído.